PrepGap – Exam Preparation Preparation Gap Finder

ACADEMIC PROJECT REPORT

Title: PrepGap - Exam Preparation Gap Finder

Author: Harsh Sahu (24BAI10561)

Institute: VIT BHOPAL UNIVERSITY

Year: 2026

VIT BHOPAL

Table of Contents

1. Abstract
2. Problem Statement & Why It Matters
3. Approach & Solution Overview
4. Key Design Decisions
5. Challenges Faced
6. What I Learned
7. System Architecture
8. Technical Implementation Details
9. Future Work & Extensions
10. Conclusion
11. Appendix – How to Run

Abstract

PrepGap is a Java based console application that helps students quantify their exam preparation. By tracking subjects, topics, completion status, and confidence scores, the tool automatically identifies missing and weak topics, producing a clear "gap report". The system persists data locally, enabling continuous progress tracking across sessions.

Problem Statement & Why It Matters

Students often experience **"illusion of competence"** – believing they understand a topic after a single read, yet lacking evidence of mastery. This leads to:

- **Fragmented coverage** of the syllabus.
- **Over confidence** on weak topics, causing poor performance.
- **Inefficient revision** where strong topics consume valuable study time.

A data driven, lightweight solution that quantifies readiness can empower students to focus on truly weak areas, improving exam outcomes.

Approach & Solution Overview

The solution is built around three core ideas:

1. **Granular Tracking** – Subjects contain topics; each topic records completion (true/false) and a confidence score (1 5).
2. **Readiness Analytics** – Compute a *Readiness Percentage* per subject and overall.
3. **Gap Detection** – Automatically list:
    - **Missing Topics** (never started).
    - **Weak Topics** (completed but confidence ≤ 2).

All data is stored in a simple pipe delimited flat file (subjects.txt) to keep the project lightweight and platform independent.

Key Design Decisions

Decision	Rationale
Console only UI	Keeps the project focused on core logic without UI overhead; ideal for a coursework deliverable.

Flat file persistence	No external database required, simplifying setup for any student.
Confidence Score (1 5)	Provides a quantitative measure of self assessment while remaining easy to input.
Modular OOP design	Separate `Subject` , `Topic` , `PrepAnalyzer` , and `FileHandler` classes for clear responsibilities and testability.

Challenges Faced

- **Data Integrity** – Ensuring topics stay linked to the correct subject when loading from the flat file required careful parsing and validation.
- **Input Validation** – Guarding against non numeric entries for confidence scores and menu choices was essential to avoid crashes.
- **User Experience** – Presenting analytics in a readable console format demanded thoughtful string formatting and alignment.

What I Learned

- Practical application of **encapsulation**, **collections**, and **file I/O** in Java.
- Designing a **simple DAO like** layer (`FileHandler`) for persistence.
- Translating a real world problem into a **data driven algorithm**.
- Importance of **robust input handling** for a smooth CLI experience.

System Architecture

```
classDiagram
    class Main {
        +startUI()
        +displayMenu()
    }
    class Subject {
        -String name
        -List~Topic~ topics
        +addTopic(Topic)
    }
    class Topic {
        -String name
        -boolean isCompleted
        -int confidenceLevel
    }
    class PrepAnalyzer {
        +calculateReadiness(Subject)
        +findWeakTopics(Subject)
        +findMissingTopics(Subject)
    }
    class FileHandler {
        +saveData(List~Subject~)
        +loadData() : List~Subject~
    }
    Main --> Subject : manages
    Subject "1" *-- "many" Topic : contains
    Main ..> PrepAnalyzer : uses
    Main ..> FileHandler : uses
```

Technical Implementation Details

- **Language**: Java 8+ (no external libraries).
- **Persistence**: subjects.txt – each line: `SubjectName|TopicName|Completed|Confidence` .
- **Core Classes**:
  - Main – CLI loop, menu handling.
  - Subject – Holds a list of Topic objects.
  -
  -
  -
-

Topic – Stores name, completion flag, confidence.

PrepAnalyzer – Business logic for readiness and gap detection.

FileHandler – Reads/writes the flat file using `BufferedReader/Writer` .

Error Handling: Extensive `try catch` blocks around user input and file operations.

Future Work & Extensions

- **GUI** – JavaFX or Swing front end for richer interaction.
- **Export Formats** – Direct PDF/HTML report generation from within the app.
- **Cloud Sync** – Remote storage (e.g., Firebase) for cross device access.
- **Study Timer** – Integrated Pomodoro timer to schedule revision sessions.
- **Analytics Dashboard** – Visual charts (e.g., using JFreeChart) for readiness trends.

Conclusion

PrepGap demonstrates how a modest Java console application can turn subjective study habits into actionable data. By surfacing missing and weak topics, it guides students toward focused revision, ultimately improving exam preparedness.

Appendix – How to Run

1. **Compile**:

   ```
   javac src/*.java -d .
   ```

2. **Execute**:

   ```
   java src.Main
   ```

3. **Data File**: Ensure `subjects.txt` resides in the project root; the program will create it on first run.